4. Der Senior-Workflow für moderne Frontend-Komponentenentwicklung

- **Framework-unabhängig** – egal ob React, Vue oder Svelte
- **Technologie-unabhängig** – egal ob Web, Desktop oder Mobile
- **Systemisches Denken** (Mental Model), das auch im Backend hilft
- Sie sind natürlich noch kein Senior – das kommt durch Berufserfahrung. Aber Sie können bereits **denken** wie einer.

4.1 Die 5 Schritte

1. Komponentenstruktur
2. HTML-Struktur
3. Datenfluss
4. Funktionalität
5. Styling

Jede Komponente wird in genau dieser Reihenfolge entwickelt.

4.2 Schritt 1: Komponentenstruktur und Hierarchie festlegen

- Welche Komponenten werden benötigt?
- Wie ist die Hierarchie?
- Welche Slots/Bereiche gibt es?

Dazu können folgende Techniken angewendet werden

- Komponentenraster
    - Screenshot von Mockup
    - oder wenn nicht vorhanden: Wireframe
- Komponentendiagramm

Tools: tldraw, Excalidraw, Figma, Balsamiq, Visio
Persönliche Empfehlung: tldraw

4.2 Schritt 1 – Beispiel: Komponentenraster

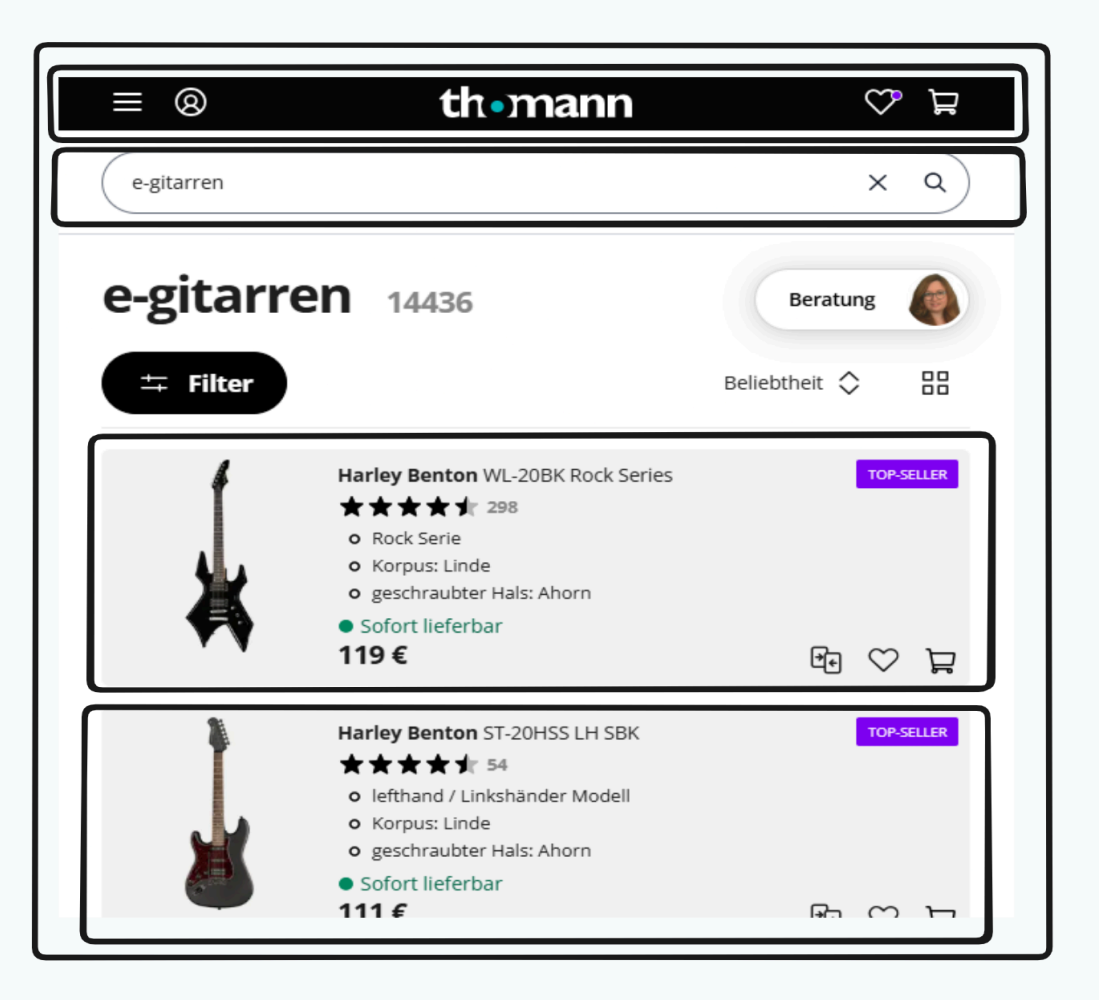


Abb. 1: UI in Komponenten zerlegen

4.2 Schritt 1 – Beispiel: Komponentenbaum

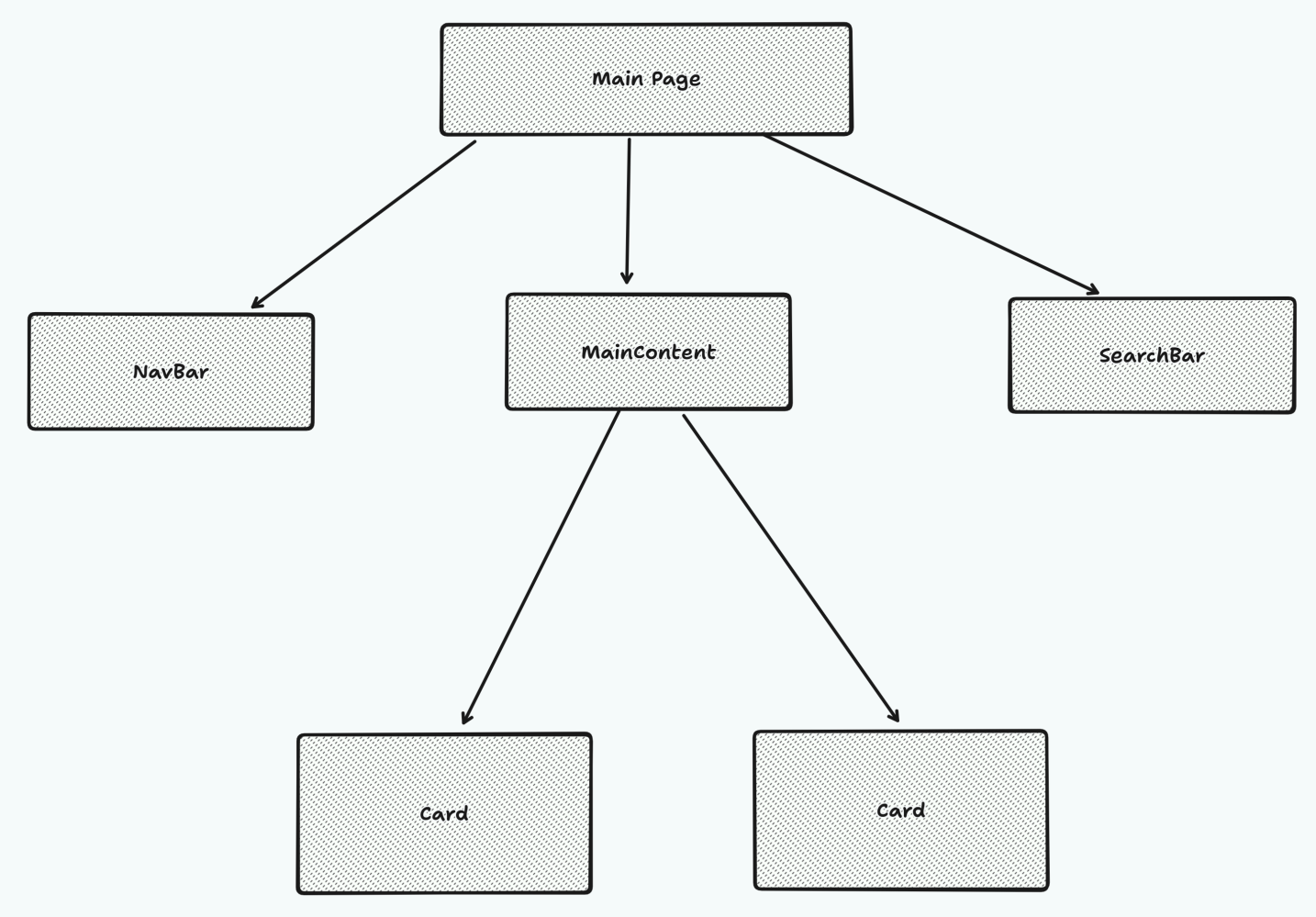
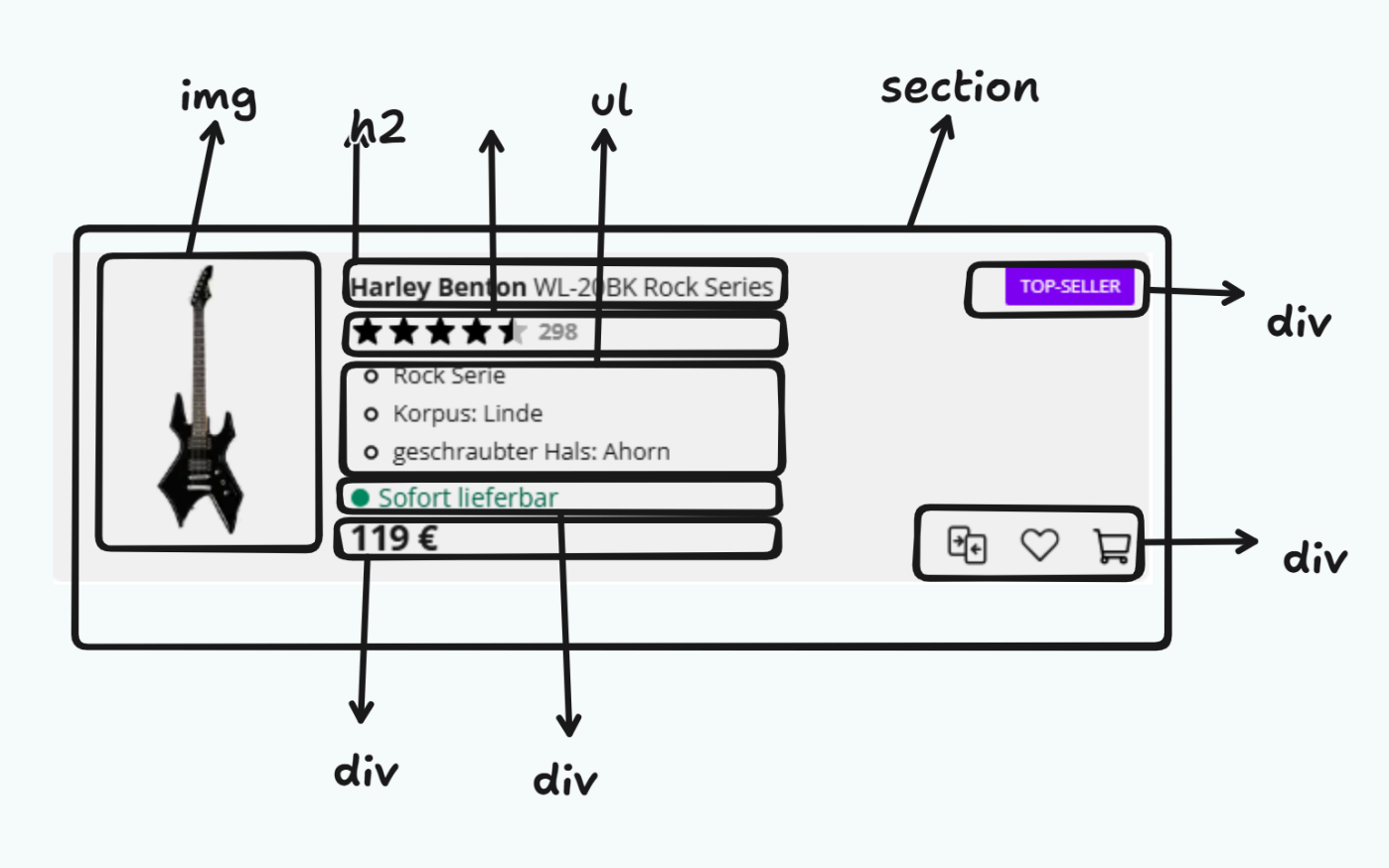


Abb. 2: Komponentenbaum

4.3 Schritt 2: HTML-Struktur

- Welche HTML-Elemente existieren? → Semantisches HTML
- Die Komponente wird wie ein Wireframe aufgebaut.



4.4 Schritt 3: Datenfluss

- Identifizieren von UI-States
  - State = „etwas, das sich ändert“
  - Wird zum Beispiel ein Bild oder ein Text ausgeblendet?
  - Essenzieller vs. abgeleiteter State
- Datenfluss skizzieren
  - Local State
  - Lifted State → Props
  - Global → injected

4.5 Schritt 4: Funktionalität

- Identifizieren von Effects, wenn sich ein State ändert
    - Beispiel: Local Storage aktualisieren
- Identifizieren von State-Transitions als Reaktion auf User Events
    - z. B. Klicks auf Buttons, Eingaben in Formularen
    - Datenlebenszyklus: Daten vom Server holen, Loading- und Error-State managen

4.6 Schritt 5: Styling

1. Layout
2. Farben
3. Responsiveness
4. Animationen

Warum Styling zuletzt? Zu frühes Styling führt oft zu:

- Ablenkung
- unnötigem Refactoring
- versteckten Architekturproblemen
- Fokusverlust

Der Senior-Workflow priorisiert Funktion vor Design.

4.7 Fazit

4.7.1 Typische Fehler ohne den Senior-Workflow

- Sofort Styling beginnen
- Zu früh abstrahieren
- State unklar verteilen
- Komponenten zu groß machen
- Logik und UI vermischen

4.7.2 Vorteile der Methode

Technisch

- bessere Wartbarkeit
- sauberer Data Flow
- bessere Testbarkeit
- klarere Komponenten

Im Team

- besseres Code Review
- einheitliche Struktur
- leichteres Onboarding

4.8 Demo: Thomann-Webseite

https://www.tldraw.com/f/6ILM5hhs_NHVaPgEUzfY4?d=v-4171.-1865.7154.3917.page